

Architectural Design Doc

Overview

This document presents the proposed cloud architecture for Company Inc., including environment structure, deployment strategy, and infrastructure design decisions. The architecture is designed to ensure scalability, reliability, and operational efficiency.

Cloud Environment Structure

Company Inc. uses **GCP** as its primary cloud provider due to its strong support for containerized workloads, simplified infrastructure management, and clear project-based isolation. Compared to AWS, GCP provides a simpler operational model for small to mid-sized engineering teams while maintaining strong security, IAM, and billing controls.

A **three-project architecture** is recommended to ensure proper environment isolation, secure deployments, and efficient resource management:

- **Shared Services Project** - hosts common infrastructure used across all environments, including Artifact Registry, Terraform state storage, and CI/CD service accounts.
- **Staging Project** - non-production environment used for testing, QA, and validating releases before production deployment. Staging mirrors production architecture but operates with reduced capacity (smaller nodes and 1-2 replicas per service) for cost-efficient testing and validation.
- **Production Project** - isolated environment hosting customer-facing workloads and production databases.

This structure provides strong separation between staging and production, reducing security risks and minimizing operational blast radius. It also enables clear cost tracking per environment and simplifies access management through project-level IAM policies.

Network Design

The system runs in a single GCP VPC deployed in the europe-west9 (Paris) region to ensure data residency compliance and low-latency access for EU users. Each environment (staging, production) has its own isolated VPC with an identical network structure.

The network is segmented into two non-overlapping subnets:

- **Public subnet** for external purposes
- **Private subnet** hosting GKE worker nodes

The GKE cluster uses a DNS-based control plane endpoint, replacing the traditional IP-based access model (public endpoint with authorized networks). Access to the Kubernetes API is secured using IAM-based authentication and Google Cloud networking controls, removing the need for bastion hosts or IP allowlists.

All inbound traffic enters through an external HTTPS Load Balancer and is routed to services running in the private subnet via Kubernetes gateway api. Private nodes use Cloud NAT for controlled outbound internet access, including container image pulls and system updates.

Firewall rules follow a deny-by-default model, allowing only required traffic between the load balancer, application layer, database tier etc

Compute Platform

The application is deployed on Google Kubernetes Engine (GKE) and managed using separate staging and production clusters to ensure strong environment isolation and independent scaling.

Cluster Strategy:

- Staging Cluster uses a simplified single-zone GKE cluster with E2 machine types. It is optimized for cost efficiency and functional validation, with minimal redundancy and reduced resource allocation.

- Production Cluster runs a private GKE cluster in the europe-west9 (Paris) region with a DNS-based control plane endpoint, removing reliance on public API exposure and IP-based authorized networks.

The cluster uses a hybrid node management model:

- Cluster Autoscaler handles scaling of nodes within existing node pools
- Node Auto-Provisioning dynamically creates and adjusts node pools and machine types based on workload demand

A minimum of three nodes is enforced to ensure proper distribution of stateful workloads such as MongoDB across separate failure domains.

Node Groups and Resource Allocation

Production uses dedicated node pools to isolate workload types:

- MongoDB runs on dedicated nodes with strict scheduling constraints (node affinity, hard anti-affinity, topology spread constraints, and PodDisruptionBudgets)
- Backend services run on general-purpose node pools
- Frontend workloads run on separate pools optimized for burst traffic patterns

Stateless workloads use soft anti-affinity and topology spread constraints to ensure even distribution without restricting scheduling flexibility.

Resource allocation follows a progressive tuning approach:

- Initial deployment runs without strict production limits for baseline metric collection
- Resource requests and limits are later defined based on observed utilization over a two-week period
- Final limits are set 15–20% above observed peak usage to ensure stability under load
- Namespace-level resource quotas enforce isolation between MongoDB and application workloads

Autoscaling Strategy:

- Horizontal Pod Autoscaling (HPA) is used for backend services based on CPU and memory utilization
- KEDA is used for frontend scaling based on ingress/request-driven metrics
- Cluster Autoscaler and Node Auto-Provisioning handle infrastructure-level scaling (nodes and machine types)

Vertical scaling is handled manually through periodic adjustment of resource requests and limits based on observability data rather than automated VPA.

Containerization and Image Management

Image Build Process

Container images are built via GitHub Actions CI pipelines triggered on merge to the main branch. The pipeline includes:

- Linting and unit tests
- SAST scanning
- Container vulnerability scanning (Trivy)
- Docker image build with immutable Git commit SHA tagging

Container Registry

All container images are stored in Google Artifact Registry within a shared services project. Separate repositories are maintained for staging and production environments. Staging images are subject to lifecycle cleanup after 180 days, while production images are retained as immutable release artifacts.

CI/CD and Deployment Strategy

Deployment follows a GitOps model using ArgoCD.

- CI (GitHub Actions) is responsible for building and pushing images
- CD (ArgoCD) manages deployment synchronization to Kubernetes clusters

- ArgoCD Image Updater monitors Artifact Registry for new image versions and automatically updates GitOps manifests, which are then reconciled by ArgoCD to trigger deployments.

This ensures a strict separation between build and deployment concerns and guarantees reproducible deployments across environments.

Database

Database Service Selection

The system uses a self-managed MongoDB deployment on Google Kubernetes Engine (GKE) using a Kubernetes-native MongoDB Community Operator and StatefulSets. This approach is chosen to maintain full control over replication topology, storage configuration, and workload scheduling within the Kubernetes environment, while avoiding dependency on external managed database services.

High Availability Strategy

MongoDB is deployed as a replica set distributed across a multi-zone GKE cluster (three availability zones). High availability is achieved through:

- Dedicated node pools for database workloads
- Hard pod anti-affinity rules to ensure replicas are never scheduled on the same node
- Topology spread constraints to enforce distribution across zones and nodes
- Node affinity rules restricting database workloads to dedicated compute resources
- PodDisruptionBudgets to preserve quorum during voluntary disruptions such as node upgrades or scaling events

This ensures continuous availability even in the event of node or zone-level failures.

Backup Strategy

Backups are handled using Kubernetes-native backup mechanisms integrated with GKE backup capabilities. MongoDB data is backed up at the volume and application level with scheduled daily snapshots.

Backup policies include:

- Automated daily backups of persistent volumes
- Retention policies with configurable retention periods (minimum six months)
- Protection against accidental deletion using backup immutability controls where supported
- Namespace-level backup scoping using ProtectedApplication or equivalent backup grouping mechanisms

Backups are stored in Google Cloud Storage with lifecycle policies aligned to compliance and retention requirements.

Disaster Recovery Strategy

Disaster recovery is based on a combination of infrastructure reproducibility and data restoration:

- Infrastructure is fully defined as code using Terraform and GitOps, enabling rapid cluster and workload reconstruction
- Application state is restored from the latest available MongoDB backups
- GitOps (ArgoCD) ensures automatic redeployment of application workloads after cluster recovery
- Typical full cluster recovery time is approximately 20 minutes for infrastructure recreation, with additional time required for database restoration depending on dataset size

This model prioritizes fast infrastructure recovery combined with reliable point-in-time data restoration as the primary recovery mechanism.